Spend Steps, Not Subsets: Time-Targeted Optimization for Online Gaussian Splatting Mapping

Anonymous ICCV submission

Paper ID *****

Abstract

Online RGB-D Gaussian Splatting mappers commonly optimize under a modeled per-Gaussian budget that decides which Gaussians to update each frame. We show that with a fast GPU rasterizer the wall-clock cost of a frame is dominated by a per-step overhead of about 5.2 ms that does not depend on how many Gaussians a step updates (a fixed 16 ms per frame remains), so selecting subsets saves only about 3 ms and costs several dB. We profile the pipeline and remove its main overheads (2.0–2.4× faster at unchanged output), update every Gaussian at every step, and control the number of steps *with a feedback loop on measured frame time. The step count is only useful with an optimizer whose statistics do not go stale between frames; short-memory Adam, higher learning rates and an in-frame decay account for* $+6.98$ *dB at equal time. On 11 test scenes and five later windows of TUM sequences, the controller meets 45–90 ms targets within 3.7% and, at the original system's frame time, raises PSNR on the frames being mapped by 10.53 dB (worst scene* $+7.41$ *dB). That gain is largely specific to the current view. A frame that arrives next improves by* $+1.9$ *to* $+3.3$ *dB at* $K{=}3$ *steps, but the* final *map on frames that were never mapped improves by only* $+0.83$ *dB at* $K{=}3$ *and by* -0.33 *dB at* $K{=}10$*, because a mapper that fits one frame at a time forgets earlier views. Replaying stored past keyframes in a third of the steps, at about 3% extra time, raises the final-map PSNR by* $+2.09$ *and* $+2.33$ *dB over the original system on the test scenes. In a real system, the tracker of MonoGS run against our map with replay reaches a final-map PSNR within 0.8 dB of original MonoGS under MonoGS's own evaluation, at 1.8× its speed and similar trajectory error (one run each, six TUM sequences). Value-gated growth, early stopping and error-proportional step allocation do not beat uniform steps. Main experiments use ground-truth poses on a laptop GPU.*

1. Introduction

Online RGB-D 3D Gaussian Splatting (3DGS) [6] underlies mapping systems such as SplaTAM [5], MonoGS [10], Photo-SLAM [4] and RTG-SLAM [12]. On robots and headsets the optimization available per streaming frame is limited, so a common design budgets it: a cost model prices the update of each Gaussian and a scheduler picks the subset that fits [11, 16]. Our earlier study of such a system, which we call the *original system*, found that the modeled budget was the bottleneck.

Here we measure wall-clock time instead of the model. On a GPU rasterizer a frame costs a fixed 16 ms plus 5.2 ms for every optimization step (median over 16 scenes and windows), almost independently of how many Gaussians a step updates. Selecting a subset therefore saves little: updating all Gaussians instead of the original system's subset costs only 3 ms more at a matched step count, and gains several dB. The right unit of compute is the *step*, and the right controller adjusts the number of steps to the measured frame time.

We build on this in three parts. (1) We profile the pipeline and remove its main overheads, making it 2.0–2.4× faster at unchanged output (Sec. 4). (2) We update every Gaussian at every step and control the step count with a feedback loop on measured frame time (Sec. 5). (3) We show that this only pays off with an optimizer whose statistics do not go stale across frames: default Adam carries about 125 frames of second-moment memory, which throttles the steps as the rendering error falls; short-memory Adam with higher learning rates and an in-frame decay is worth $+6.98$ dB at equal time.

At about the original system's frame time (44 ms), the time-targeted system raises PSNR on mapped frames by $+10.53$ dB on average over 11 test scenes (worst scene $+7.41$ dB) and by $+9.90$ dB on five later windows. The controller keeps the measured time within 3.7% of 45–90 ms targets and matches the best uniform step count at equal time (± 0.2 dB), whereas one fixed step count varies from 53 to 77 ms across scenes. These are large numbers, and a reader should ask whether the original system was simply

under-tuned. It partly was: with the new optimizer but its old scheduler it gains only $+3.44$ dB, so most of the gain needs both the optimizer and the removal of subset selection (Tab. 2). The gain on mapped frames overstates map quality. For a frame that arrives next it is $+1.9$ to $+3.3$ dB at three steps and saturates (Tab. 4); for the final map on frames that were never mapped it is only $+0.83$ dB at three steps and -0.33 dB at ten, because a mapper that fits one frame at a time forgets earlier views (Tab. 5). Replaying stored keyframes in a third of the steps repairs this for about 3% more time: $+2.09$ dB over the original system on the test scenes at three steps.

Contributions. (i) A wall-clock cost analysis showing that per-step overhead, not the number of updated Gaussians, determines frame time, with core changes that make the pipeline about $2\times$ faster at identical output. (ii) A time-targeted step controller and an optimizer setting that make the added steps useful, with all constants set on four development scenes. (iii) A matched-time evaluation on 11 scenes and five fresh windows against a uniform-step curve, an optimizer ablation, a held-out and retention analysis showing that one-frame-at-a-time mapping forgets earlier views and that keyframe replay repairs this, an integration with the tracker of MonoGS, and negative results for value-gated growth, early stopping and error-proportional allocation.

2. Related Work

Online 3DGS SLAM and dense mapping. GS-SLAM [19] and SplaTAM [5] introduced differentiable 3D-Gaussian SLAM, with SplaTAM using silhouette-guided optimization; Gaussian-SLAM [20] adds a sub-map design and Loop-Splat [22] adds loop closure by registering Gaussian submaps; Point-SLAM [13] uses dynamic neural point clouds; MonoGS [10] and Photo-SLAM [4] push toward real-time operation with loop closure; RTG-SLAM [12] adds compact opaque/transparent roles with rule-based stable/unstable selection and is our primary external policy baseline (re-implemented under exactly matched budgets, since native-system comparison would confound renderer, tracker, and scheduler effects). Unlike these systems, our object of study is not the full SLAM stack but the compute-allocation layer inside it; we evaluate mapping under ground-truth poses with tracking explicitly decoupled (Sec. 7). **Scalable and efficient Gaussian mapping.** Recent systems address online map scalability through representation and update design. VTGaussian-SLAM [2] ties Gaussians to RGB-D views and avoids keeping every primitive learnable, targeting large scenes and memory use. SEGS-SLAM [18] improves structure and appearance consistency, while DyGS-SLAM [3] focuses on dynamic-scene localization and reconstruction. In parallel, SteepGS [17] derives compact density-control updates for 3DGS reconstruction. A recent online-mapping preprint studies geometry-aware initialization and densification under short optimization schedules [8]. These methods make clear that map representation, density control, and online optimization are coupled. Our study asks how growth rate and update policy should respond to coverage and observable scene statistics under a fixed per-frame budget and Gaussian representation. We do not claim density control or efficient Gaussian mapping as new in general. The per-scene SOTA numbers in Appendix Supp. Tab. S2 are protocol-mismatched context. **Density control for 3DGS.** Taming 3DGS [9] uses guided, constructive densification under resource limits; Mini-Splatting [1] targets a constrained Gaussian count; Pixel-GS [21] reweights the densification gradient by covered pixels; and 3DGS-MCMC [7] replaces heuristic densification and pruning with relocation. These methods address offline reconstruction. Our setting differs in that a per-frame optimization budget is shared between existing and newly spawned Gaussians. We do not run them (Sec. 7), but compare against a size-matched Gaussian-count budget in the supplement. **Evaluation protocols in dense SLAM.** Published systems report full-sequence train-view rendering with global refinement at full resolution (Replica 33–42 dB for MonoGS/Point-SLAM/RTG-SLAM; TUM fr2_xyz 16–25 dB for MonoGS/SplaTAM/Photo-SLAM) plus full-trajectory ATE. Our protocol is deliberately narrower — budgeted online mapping on fixed horizons — so cross-paper numbers are contextual range, never superiority claims (Supp. Tab. S2). **Budgeted selection.** Our earlier work and the original system studied here choose which Gaussians to update under a modeled per-Gaussian budget [11, 16]. We show that, once the renderer is fast, this selection saves little wall-clock time.

3. Setting and Protocol

Mapper. Per frame the mapper renders the current map, forms per-pixel color and depth residuals, spawns Gaussians where the residual is high (at most 500 per frame), runs K optimization steps (Adam on color L1, SSIM and depth L1, sparse-aware state per Gaussian), and prunes. The *original system* runs at most $K{=}5$ steps and chooses, by a fractional knapsack on raw rendering error, which Gaussians receive how many of them so that a modeled cost stays below $B{=}15$ ms. The rasterizer is `gsplat`. Poses are ground truth; a closed-loop tracker check is in the supplement.

Timing. Frame time is the CUDA-synchronized latency of one `process_frame` call on a laptop GPU, including per-frame evaluation (about 4 ms), at 320×240. It excludes tracking.

Data and protocol. TUM RGB-D [15] and Replica [14]; two seeds. Constants were set on four *development* scenes (`fr1_desk`, `fr3_structure_texture_near`, `replica_room1`, `replica_office2`). The *test* set is 11 other scenes (150 frames for `fr2_xyz`, `fr1_xyz`; 99 for the rest, as the local Replica copies have 100–120 frames),

Scene	ms before	ms after	speed-up	PSNR before	PSNR after
fr2_xyz	86	41	2.1×	26.00	26.08
fr1_xyz	105	44	2.4×	22.02	22.01
office0	90	45	2.0×	30.61	30.59

Table 1. **Core speed-ups at unchanged output.** Original budgeted configuration on the code revision before and after the changes of Sec. 4; end-to-end frame time in ms (including per-frame evaluation), mean PSNR (dB), two seeds, 100 frames.

and five *fresh* windows of 150 later frames of long TUM sequences. The observation that steps matter first appeared while profiling fr2_xyz, a test scene; the development scenes and fresh windows limit this exposure but do not remove it. For *held-out views*, every eighth frame is rendered but never mapped.

Equal-time comparison. Methods differ in frame time, so we compare each method with the *uniform-K curve*: the original mapper with all Gaussians active and $K \in \{3, 6, 10, 15\}$ steps per frame and the same optimizer, linearly interpolated in measured time at the method's own mean time.

4. Where the Time Goes

Fitting frame time as $a + bK$ over $K \in \{3, 6, 10, 15\}$ gives a median a=16 ms and b=5.2 ms per step (range 4.1–5.9); at the original K=5, 39% of the time is the fixed part. GPU profiling at K=8 showed three avoidable costs, which we removed without changing the computation:

SSIM windows. The 11×11 Gaussian window was applied as a dense depthwise convolution whose backward pass took a third of GPU time; the window is separable, so we apply it as two 1-D passes (value and gradient agree to 10^{-6}). *Covariance.* $\Sigma = RSS^\top R^\top$ via batched 3×3 GEMMs cost about 20 ms per frame in cuBLAS; the same product as broadcast elementwise operations is several times cheaper in forward and backward. *Indexing.* With every Gaussian active, the gather of parameters, the scatter of gradients and the optimizer's gather/scatter of moments are pure overhead; a dense path skips them (equal to the sparse path to 10^{-6} in tests). A per-Gaussian device synchronization in the state store was vectorized.

Together these make the original configuration 2.0–2.4× faster at the same PSNR (Tab. 1); they apply to every method below.

5. Time-Targeted Steps

Uniform steps. All Gaussians are active at every step, and the modeled budget is held at a large constant so that it never binds. The number of steps becomes the compute knob.

Time targeting. With frame time τ_t and target T, the step count n follows $n \leftarrow \mathrm{clip}\big(n\,(T/\hat{\tau})^\gamma,\, n_{\min},\, n_{\max}\big)$, where

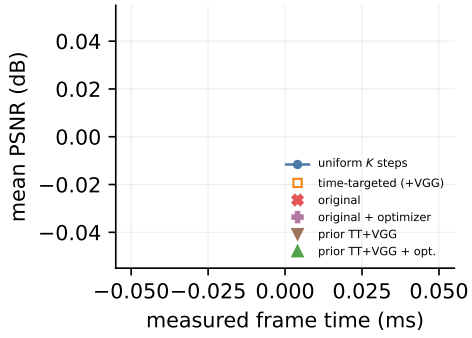


Figure 1. Mean PSNR versus measured frame time on the 11 test scenes. The original system lies far below the uniform-K curve; the time-targeted controller sits on the curve at the requested time.

$\hat{\tau}$ is an exponential average of τ_t (weight 0.5), γ=0.5, $n \in [2, 24]$ and the integer used is $\mathrm{round}(n)$. A compact map or a fast scene automatically receives more steps, a large map fewer.

Short-memory optimizer. Adam's second-moment estimate with β_2=0.999 remembers about 125 frames at 8 steps per frame, while the rendering error shrinks as the map fits; stale large estimates then keep steps small. We use $(\beta_1, \beta_2) = (0.5, 0.8)$, learning rates ×6 (scale ×12) of the original ones, and a linear decay within each frame from 1.4× to 0.6×. These came from coordinate search on the four development scenes (Sec. 6); they are one setting for all scenes and both datasets. Value-gated growth, an additional controller from our earlier version that throttles spawning when recently spawned Gaussians are not worth keeping, is described in the supplement and is optional here.

Keyframe replay. Fitting one frame at a time with many steps overwrites earlier views. Every third step (steps $3, 6, 9, \ldots$ of a frame) instead optimizes a past frame drawn uniformly from a reservoir of 32 earlier mapped frames (every second mapped frame is offered to the reservoir; uniform reservoir sampling keeps a sample of the whole history), rendered at its stored pose with the same loss. It costs no extra renders per step, only 3% more time in our measurements. The three constants were chosen on the four development scenes and are insensitive: replaying every second step, or a reservoir of 16 or 64, or offering every frame or every third, change final-map PSNR by under 0.4 dB.

6. Experiments

Against the original system (Tab. 2, Fig. 1). At matched frame time the time-targeted system with T=45 ms ends at 46 ms against 44 ms and is +10.53 dB higher on average, +7.41 dB on the worst scene, and +9.90 dB on the fresh windows. Even K=3 steps, at 32 ms, are +7.83 dB. At matched time the original system is −10.27 dB relative

	11 test scenes				5 fresh windows		
Method	ms	ΔPSNR mean	median	worst	ms	ΔPSNR mean	vs. uniform K at equal time (test)
Original budgeted system	44	0.00	0.00	0.00	42	0.00	−10.27
+ steps-regime optimizer	44	+3.44	+3.43	+1.81	42	+3.50	−6.78
Prior TT+VGG controllers	43	+2.59	+2.91	+0.01	38	+1.19	−7.36
Prior TT+VGG + optimizer	43	+10.13	+10.39	+7.42	39	+9.07	+0.13
Uniform K=3 steps	32	+7.83	+8.03	+5.54	29	+6.48	—
Uniform K=6 steps	47	+10.88	+10.92	+8.07	43	+9.92	—
Uniform K=10 steps	68	+12.70	+12.94	+9.86	62	+11.75	—
Uniform K=15 steps	94	+13.89	+14.16	+11.17	85	+12.50	—
Time-targeted steps, T=60	60	+12.11	+12.38	+9.04	60	+11.38	+0.08
+ VGG, T=45	46	+10.53	+10.69	+7.41	45	+9.90	+0.03
+ VGG, T=60	60	+12.01	+12.33	+8.91	60	+11.29	−0.02
+ VGG, T=75	74	+12.83	+13.44	+9.90	74	+11.89	−0.17
+ VGG, T=90	89	+13.44	+13.87	+10.69	89	+12.32	−0.11

Table 2. **Steps regime versus the original budgeted system.** PSNR change in dB on the mapped frames relative to the original system (unthrottled raw-error selection, 5 steps, modeled 15 ms budget) on the same code; mean measured end-to-end frame time in ms. The last column is the mean PSNR difference to the uniform-K curve (linear interpolation in measured time) at the method's own mean time; the original system lies far below that curve. Two seeds per scene; "test" are the 11 scenes of the earlier protocol, "fresh" are five later windows of long TUM sequences. All optimizer constants were set on four development scenes.

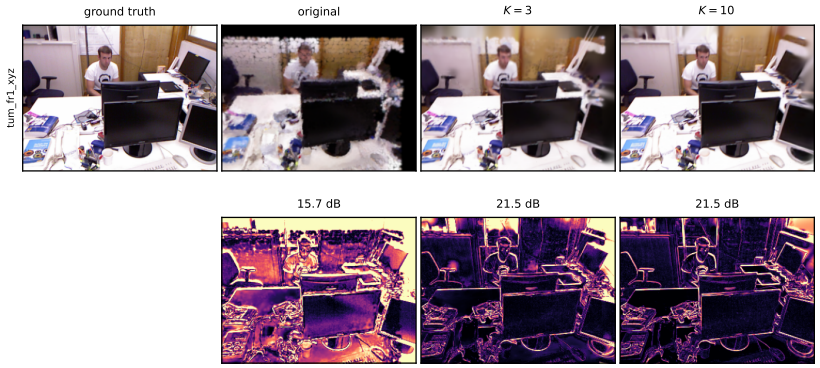


Figure 2. **Held-out view** (`fr1_xyz`, mapped frames 1–80, the next frame rendered but never mapped). Top: renders; bottom: absolute color error with PSNR. Three steps already remove most of the original system's holes and blur; ten steps add little. A second scene is in the supplement.

to the uniform-K curve. Applying the new optimizer to the original scheduler gains +3.44 dB, still far below the curve; applying it to the earlier time-targeted budget controllers (supplement) reaches the curve (+0.13 dB), because there the budget grows until every Gaussian is updated. The subset scheduler is therefore not what limits quality once the budget is large; it is what keeps the original system off the curve.

Time targeting. For $T \in \{45, 60, 75, 90\}$ ms the mean measured time is within 0.9% of the target (at most 3.7%) across 16 scenes and windows. A fixed K=10 varies from 53 to 77 ms across the same units, while the controller at T=75 stays within 73–75 ms. At equal time it is neither better nor worse than the uniform-K curve (0.08 dB at T=60 without value-gated growth). Its value is meeting a latency target without per-scene calibration, not extra quality.

Optimizer ablation (Tab. 3). Removing the short-memory betas costs 4.03 dB at K=10, the learning rates 4.53 dB and the in-frame decay 1.89 dB; removing all three costs 6.98 dB. The step count only pays with the right optimizer.

Next-frame held-out views (Tab. 4). All three configurations improve held-out PSNR on every scene, but by much less than mapped-frame PSNR: K=3 keeps 36–41% of its mapped gain and K=10 only 24–29%. Moving from K=3 to K=10 adds about 0.5 dB held out but 4–5 dB on mapped frames. These frames arrive one step after the last mapped frame, so this measures next-frame prediction; it does not

Optimizer	fr2	fr1	off0	off1	mean	Δ
full	37.61	35.28	41.97	43.76	39.65	0.00
default betas	32.59	30.58	38.81	40.52	35.62	−4.03
default lr	34.22	31.21	36.76	38.32	35.13	−4.53
no lr decay	35.14	32.64	40.85	42.42	37.76	−1.89
none of three	30.24	27.26	35.93	37.26	32.68	−6.98

Table 3. **Optimizer ablation** at uniform $K{=}10$ steps (about 68 ms), mean PSNR (dB) on mapped frames, two seeds, four test scenes (fr2, fr1: TUM xyz sequences; off0, off1: Replica).

		fr2	fr1	off0	off1
original	mapped	24.8	21.5	30.2	33.7
	held-out	24.0	20.5	26.9	30.9
$K{=}3$	Δ mapped	+7.8	+8.1	+7.2	+5.5
	Δ held-out	+2.9	+3.3	+2.9	+1.9
$K{=}10$	Δ mapped	+12.5	+13.5	+11.3	+9.8
	Δ held-out	+3.5	+3.6	+3.3	+2.3
TTS, $T{=}75$	Δ mapped	+12.9	+13.7	+11.5	+10.0
	Δ held-out	+3.4	+3.5	+3.2	+2.4

Table 4. **Held-out views.** Every eighth frame is rendered but never mapped. PSNR (dB) of the original system and the change for three configurations; two seeds. Gains on held-out frames are positive on every scene but a fraction of the gain on mapped frames.

	11 test scenes			5 fresh windows		
Method	next	final	early	next	final	early
original system	21.8	23.2	22.8	20.9	18.5	17.5
$K{=}3$ steps	24.8	24.0	23.1	23.4	18.0	16.9
$K{=}3$ + replay	24.3	25.3	24.6	22.7	19.8	19.3
$K{=}10$ steps	25.4	22.9	22.0	24.5	17.3	15.9
$K{=}10$ + replay	25.4	25.5	24.8	24.1	20.3	20.3

Table 5. **Keeping earlier views.** Mean PSNR (dB) with ground-truth poses; frames with index divisible by 5 are never mapped. next: those frames when they arrive; final: the final map on those frames; early: the final map on mapped frames of the first third of the sequence. Two seeds. Keyframe replay (every third step uses a stored past frame) is the last-listed addition.

Scene	orig. dB	ours dB	orig. ATE	ours ATE
`fr2_xyz`	27.4	37.6	7.3 cm	14.0 cm
`fr1_xyz`	23.6	35.3	6.4 cm	5.3 cm
`fr1_rpy`	21.6	35.5	12.9 cm	11.6 cm
`fr3_sitting_static`	29.6	37.9	13.8 cm	19.4 cm
`fr1_room`†	14.9	24.7	154.8 cm	166.6 cm

Table 6. **Closed-loop map-based tracking in the steps regime**, three seeds, TUM, "ours" = time-targeted steps at 60 ms. PSNR is measured at the tracked pose. †Tracking fails for both (excluded from the summary).

test whether the map retains the scene (next paragraph) or new trajectories.

Keeping earlier views (Tab. 5). MonoGS evaluates the *final* map on frames that were not mapped. Doing the same with every fifth frame held out, uniform steps help little: $K{=}3$ changes final-map PSNR by $+0.83$ dB over the original system on the test scenes (better on 8 of 11) and $K{=}10$ by -0.33 dB; on the five fresh windows the changes are -0.48 dB and -1.20 dB. Even frames mapped in the first third of the sequence are rendered by the final map at only 23.07 dB at $K{=}3$. Keyframe replay raises final-map PSNR over the same step count on every scene and window ($+1.26$ dB at $K{=}3$ and $+2.66$ dB at $K{=}10$ on the test scenes; $+1.81$ and $+3.03$ dB on the fresh windows) and exceeds the original system on 10 (at $K{=}3$) and 11 (at $K{=}10$) of 11 test scenes. It lowers next-frame PSNR slightly (24.34 against 24.76 dB at $K{=}3$).

Closed-loop tracking (Tab. 6). With poses from a simple map-based tracker that aligns each frame to the current map (supplement), the time-targeted system raises PSNR at the tracked pose by $+8.4$ to $+14.0$ dB on four TUM scenes, but trajectory error does not improve reliably: it changes by -1.3 to $+6.7$ cm and is worse on 2 of the four scenes. A map fitted tightly to a drifting trajectory is a worse alignment target, so better map fit does not imply better tracking.

MonoGS tracking with our mapper (Tab. 7). To test the mapper inside a real system, we run the pose tracker of MonoGS [10] (100 photometric plus depth iterations per frame, its own rasterizer) against the map built by our mapper, which receives the estimated pose; MonoGS's keyframe mapping, window optimization and loop handling are not used. We evaluate as MonoGS does: the final map on every fifth frame, here none of which is mapped, and ATE over every fifth frame, on the first 150 frames of six TUM sequences at $640{\times}480$ and one run per entry. Without replay our mapper is well below MonoGS (15.69 dB with the original mapper, 14.51 dB at $K{=}3$, against 19.73 dB), as expected from the retention analysis. With replay, $K{=}3$ reaches 18.91 dB, 0.8 dB below MonoGS on average, and exceeds it on 1 of 6 sequences (best $+3.4$ dB, worst -4.0 dB), at 661 ms per frame against 1176 ms for MonoGS ($1.8\times$ faster) and a mean trajectory error of 1.29 cm against 1.16 cm. Most of the per-frame time is tracking, which our mapper does not change. MonoGS's own mapping is therefore not beaten in quality here; the result is comparable quality at a lower time per frame. The two systems' evaluation frames differ in one respect (MonoGS skips its keyframes), and timing varied between MonoGS runs on the same machine.

Relation to our earlier analysis. The earlier version of this work argued that a fixed modeled budget dilutes as the map grows and that raising it in proportion to map size improves quality on 15 scenes. Those results, and controllers that scale the modeled budget or throttle spawning, are in the supplement (section "Earlier analysis of the original

	MonoGS		+ original mapper		+ ours K=3		+ ours K=3, replay		+ ours K=10, replay	
Sequence	dB	cm	dB	cm	dB	cm	dB	cm	dB	cm
`fr1_desk`	18.7	1.35	13.9	1.57	12.5	1.30	16.3	1.38	17.4	1.40
`fr2_xyz`	20.4	0.70	15.6	0.59	12.2	0.38	19.7	0.51	19.8	0.46
`fr1_xyz`	20.6	1.06	19.9	1.08	16.3	1.44	20.5	1.02	19.8	1.10
`fr1_rpy`	12.6	2.56	14.8	3.39	13.3	3.49	16.0	3.24	15.3	3.81
`fr2_desk`	24.0	0.69	18.8	0.97	14.2	1.03	20.1	0.91	20.7	1.11
`fr3_sitting_static`	22.0	0.62	11.3	0.63	18.6	0.86	20.9	0.67	21.3	0.75
mean	19.7	1.16	15.7	1.37	14.5	1.41	18.9	1.29	19.0	1.44
ms/frame	1176		593		674		661		861	

Table 7. **MonoGS's tracker with our mapper, under MonoGS's own evaluation.** First 150 frames of six TUM sequences at 640×480, one run per entry. dB: PSNR of the final map on every fifth frame (MonoGS: those that are not keyframes; ours: all of them, none was mapped), rendered at the estimated pose. cm: Umeyama-aligned ATE RMSE over every fifth tracked frame. Per-frame time is MonoGS's total, or tracking plus mapping.

budgeted system); they are consistent with Sec. 4 (the budget was starving the optimizer) but they measured a regime that the faster core and the new optimizer make obsolete.

What did not help. Evaluated on the development scenes with a single seed (details and bundles in the supplement), none of the following beat uniform steps at equal time on the next-frame measure: value-gated growth (-0.17 dB at T=75 on test, negative on fresh windows), convergence-gated early stopping (-0.15 to -0.29 dB), error-proportional step allocation across frames (-0.32 and -0.70 dB for either sign). (Keyframe replay also left next-frame held-out PSNR almost unchanged, $+0.23$ dB, but that measure cannot show retention; the previous paragraphs show it helps.) Loss-weight, spawn-rate and densification-threshold sweeps were flat.

7. Limitations

Scope. Poses are ground truth in the main results; closed-loop checks with a simple map-based tracker and with the tracker of MonoGS are TUM only (the local Replica copies are sampled too sparsely for any tracker), use one run per setting for MonoGS, and do not include MonoGS's keyframe mapping, bundle adjustment or loop closure. We use 320×240 frames, up to 150 frames per run, a laptop GPU and two seeds that measure repeatability, not scene variance; frame times of 30–90 ms are 11–30 FPS, so the high-quality settings are not real-time at 30 FPS. **Generalization.** Mapped-frame PSNR is dominated by fitting the current view; next-frame gains are 24–41% of it, and uniform steps without replay do not improve the final map on never-mapped frames. With replay our mapper still trails MonoGS's own mapping in final-map PSNR on 5 of 6 sequences, and replay constants were chosen on four development scenes only. The next-frame, retention and MonoGS evaluations use different frame sets and protocols and should not be added together. **Baselines.** The original system was strongly under-optimized; we show the ladder from it to the new system, but we do not run offline density-control methods [1, 7, 9, 21], and only the tracker of a full SLAM system is integrated. **Exposure.** The development scenes were used to set every constant, but the key observation was first made on a test scene, and constants for value-gated growth are inherited from the earlier version and were not retuned.

8. Conclusion

When the renderer is fast, the cost of a frame is set by the number of optimization steps, not by how many Gaussians a step updates. Updating all Gaussians, controlling the number of steps from measured frame time, and using an optimizer with short memory gives $+10.53$ dB on mapped frames at the original system's frame time, but only a small gain in how well the final map renders frames it never saw, because one-frame-at-a-time mapping forgets earlier views. Replaying stored keyframes in a third of the steps recovers this for 3% more time ($+2.09$ dB over the original system), and with MonoGS's tracker running against our map the result is within 0.8 dB of MonoGS's own mapping at 1.8× the speed. Matching or beating MonoGS's keyframe-window mapping in quality at this speed is the next problem.

References

[1] Guangchi Fang and Bing Wang. Mini-Splatting: Representing scenes with a constrained number of gaussians. In *ECCV*, 2024.

[2] Pengchong Hu and Zhizhong Han. VTGaussian-SLAM: RGBD SLAM for large scale scenes with splatting view-tied 3D gaussians. In *International Conference on Machine Learning (ICML)*, pages 24347–24370, 2025.

[3] Xinggang Hu, Chenyangguang Zhang, Mingyuan Zhao, Yuanze Gui, Xiangkui Zhang, and Xiangyang Ji. DyGS-SLAM: Real-time accurate localization and gaussian reconstruction for dynamic scenes. In *ICCV*, pages 9561–9571, 2025.

[4] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photo-

realistic mapping for monocular, stereo, and RGB-D cameras. In *CVPR*, 2024.

[5] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplaTAM: Splat, track & map 3D gaussians for dense RGB-D SLAM. In *CVPR*, pages 21357–21366, 2024.

[6] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023.

[7] Shakiba Kheradmand et al. 3D gaussian splatting as Markov chain Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] Thai Luu, Quan Tran, Hieu Phan, and Tuan Dang. Geometry-aware online mapping for 3D gaussian splatting SLAM. *arXiv preprint arXiv:2608.14902*, 2026.

[9] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3DGS: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.

[10] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian splatting SLAM. In *CVPR*, pages 18039–18048, 2024.

[11] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.

[12] Zhexi Peng, Tianjia Shao, Yong Liu, Jingke Zhou, Yin Yang, Jingdong Wang, and Kun Zhou. RTG-SLAM: Real-time 3D reconstruction at scale using gaussian splatting. *ACM SIGGRAPH Conference Papers*, 2024.

[13] Erik Sandström, Yue Li, Luc Van Gool, and Martin R. Oswald. Point-SLAM: Dense neural point cloud-based SLAM. In *ICCV*, pages 18433–18444, 2023.

[14] Julian Straub, Thomas Whelan, et al. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

[15] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[16] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.

[17] Peihao Wang et al. Steepest descent density control for compact 3D gaussian splatting. In *CVPR*, pages 26663–26672, 2025.

[18] Tianci Wen, Zhiang Liu, and Yongchun Fang. SEGS-SLAM: Structure-enhanced 3D gaussian splatting SLAM with appearance embedding. In *ICCV*, pages 28103–28113, 2025.

[19] Chi Yan, Delin Qu, Dan Xu, Bin Zhao, Zhigang Wang, Dong Wang, and Xuelong Li. GS-SLAM: Dense visual SLAM with 3D gaussian splatting. In *CVPR*, pages 19595–19604, 2024.

[20] Vladimir Yugay, Yue Li, Theo Gevers, and Martin R. Oswald. Gaussian-SLAM: Photo-realistic dense SLAM with gaussian splatting. *arXiv preprint arXiv:2312.10070*, 2023.

[21] Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-GS: Density control with pixel-aware gradient for 3D gaussian splatting. In *ECCV*, 2024.

[22] Liyuan Zhu, Yue Li, Erik Sandström, Shengyu Huang, Konrad Schindler, and Iro Armeni. LoopSplat: Loop closure by registering 3D gaussian splats. In *International Conference on 3D Vision (3DV)*, 2025.